

Code Explanation

DLT Pipeline Code Explanation

This code is a Databricks Delta Live Tables (DLT) pipeline written in Python. It defines a series of tables that are used to load, transform, and aggregate customer and order data from source files.

The pipeline consists of four main tables: `customer`, `order`, `ordersummary`, and `customeraggregatespend`. Each table is defined using the `@dlt.table` decorator, which specifies the name and comment for the table.

Overview of the Code

The code is designed to load customer and order data from CSV files, perform data quality checks, and create a Slowly Changing Dimension (SCD) Type 2 table that tracks changes to customer and order data over time. The pipeline also aggregates customer spending by name and date.

Step 1: Loading Customer Data

This step loads customer data from a CSV file into a DLT table named `customer`. The data is read using Spark's `csv` method, and null values are dropped. Duplicate records are also removed based on the `CustId` column.

```
@dlt.table(
    name="customer",
    comment="Customer data from source files"
)
def customer():
    return (
        spark.read
        .option("header", "true")
        .option("inferSchema", "true")
        .csv("/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customer
data")
        .na.drop()
        .dropDuplicates(["CustId"])
    )
```

Step 2: Loading Order Data

This step loads order data from a CSV file into a DLT table named `order`. The data is read using Spark's `csv` method, and null values are dropped. Duplicate records are removed based on the `OrderId` column. The `TotalAmount` column is also calculated by multiplying the `PricePerUnit` and `Qty` columns.

```
@dlt.table(
    name="order",
    comment="Order data from source files with TotalAmount calculate
d"
)
def order():
    return (
        spark.read
        .option("header", "true")
```

```

        .option("inferSchema", "true")
        .csv("/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdat
a")
        .na.drop()
        .dropDuplicates(["OrderId"])
        .withColumn("TotalAmount", col("PricePerUnit") * col("Qty"))
    )

```

Step 3: Creating the SCD Type 2 Table

This step creates an SCD Type 2 table named `ordersummary` by joining the `customer` and `order` tables on the `CustId` column. The resulting table includes SCD Type 2 fields such as `IsActive`, `StartDate`, and `EndDate`.

```

@dlt.table(
    name="ordersummary",
    comment="SCD Type 2 table joining customer and order data",
    table_properties={
        "delta.enableChangeDataFeed": "true"
    }
)
def ordersummary():
    customer_df = dlt.read("customer")
    order_df = dlt.read("order")
    joined_df = customer_df.join(order_df, "CustId", "inner").select
    (
        "CustId",
        "Name",
        "EmailId",
        "Region",
        "OrderId",
        "ItemName",
        "PricePerUnit",
        "Qty",
        "Date"
    )
    result_df = joined_df
    .withColumn("IsActive", lit(True))
    .withColumn("StartDate", current_timestamp())
    .withColumn("EndDate", lit(None).cast("timestamp"))

```

Step 4: Handling SCD Type 2 Operations

This step checks if the `ordersummary` table already exists. If it does, the code performs SCD Type 2 operations such as expiring old records, creating new active records, and identifying unchanged records.

```

try:
    existing_df = dlt.read("ordersummary")
    changed_records = existing_df.alias("existing").join(
        joined_df.alias("new"),
        (col("existing.CustId") == col("new.CustId")) &
        (col("existing.OrderId") == col("new.OrderId")) &
        col("existing.IsActive"),
        "inner"
    )

```

```

).where(
    (col("existing.Name") != col("new.Name")) |
    (col("existing.EmailId") != col("new.EmailId")) |
    (col("existing.Region") != col("new.Region")) |
    (col("existing.ItemName") != col("new.ItemName")) |
    (col("existing.PricePerUnit") != col("new.PricePerUnit")) |
    (col("existing.Qty") != col("new.Qty")) |
    (col("existing.Date") != col("new.Date")))
).select("existing.CustId", "existing.OrderId")

expired_records = existing_df.join(
    changed_records,
    ["CustId", "OrderId"],
    "inner"
).withColumn("IsActive", lit(False))
.withColumn("EndDate", current_timestamp())

new_active_records = joined_df.join(
    changed_records,
    ["CustId", "OrderId"],
    "inner"
).withColumn("IsActive", lit(True))
.withColumn("StartDate", current_timestamp())
.withColumn("EndDate", lit(None).cast("timestamp"))

new_records = joined_df.join(
    existing_df.select("CustId", "OrderId").dropDuplicates(),
    ["CustId", "OrderId"],
    "left_anti"
).withColumn("IsActive", lit(True))
.withColumn("StartDate", current_timestamp())
.withColumn("EndDate", lit(None).cast("timestamp"))

unchanged_records = existing_df.join(
    changed_records,
    ["CustId", "OrderId"],
    "left_anti"
)

return unchanged_records.union(expired_records).union(new_active_records).union(new_records)
except Exception as e:
    return result_df

```

Step 5: Aggregating Customer Spending

This step aggregates customer spending by name and date using the `ordersummary` table. The `TotalAmount` column is calculated if it doesn't exist, and the data is filtered to include only active records.

```

@dlt.table(
    name="customeraggregatespend",
    comment="Aggregated customer spending by name and date"
)
def customeraggregatespend():
    order_summary_df = dlt.read("ordersummary")
    if "TotalAmount" not in order_summary_df.columns:

```

```
    order_summary_df = order_summary_df.withColumn(
        "TotalAmount",
        col("PricePerUnit") * col("Qty")
    )
return order_summary_df
    .filter(col("IsActive") == True)
    .groupBy("Name", "Date")
    .agg({"TotalAmount": "sum"})
    .withColumnRenamed("sum(TotalAmount)", "TotalAmount")
```